Real-Time Network Intrusion Detection System (NIDS) using Packet Inspection and Web-Based Alerting

---

MINI PROJECT SYNOPSIS

Submitted in partial fulfilment of the requirement for the award of the degree of

Bachelor of Technology

in

Computer Science & Engineering

Submitted to

DR. A.P.J. ABDUL KALAM TECHNICAL UNIVERSITY, UTTAR PRADESH, LUCKNOW

Submitted by

Student Name: Rohan Pandey
Roll No: 2501720100117
Section: 2D

Under the Supervision of

Mr. Vikas
Assistant Professor

Department of Computer Science & Engineering

Lloyd Institute of Engineering & Technology

Plot No. 3, Knowledge Park II, Greater Noida, Uttar Pradesh 201306

Session: 2026–27

INDEX

Sr. No.	Topic	Page No.
01	Introduction	3
02	Statement of Problem with Objective	5
03	Literature Survey	6
04	Expected Outcome / Scope of the Project	7
05	Tentative Work Plan (mapped to the 10-week timeline)	8
06	Software/Hardware required for the development of the project	9
07	References	10

Student's Signature **Guide's Signature**

1. Introduction

Computer networks now carry almost everything a student, a laboratory or a small office does, and the links that deliver web pages and video calls also deliver attacks. An intruder who reaches a local network can probe every host for open ports, exhaust a server with half-open connections, or place a laptop between two devices by forging address information. Each of these actions leaves a recognisable pattern in the packets crossing the network, so a program that inspects traffic closely enough can notice them as they happen.

An Intrusion Detection System (IDS) monitors activity for signs of unauthorised or abnormal behaviour and reports it to an administrator [2], [11]. A Network-based IDS (NIDS) observes packets on a network segment, whereas a Host-based IDS examines the logs of a single machine. Detection is signature-based when traffic is matched against known malicious patterns, and anomaly-based when a model of normal behaviour raises an alarm on deviation [14]. An IDS is a passive sensor; unlike a firewall it blocks nothing, which makes it safe to run on a live network.

This project, which lies in the domain of cybersecurity and computer networks, proposes a lightweight real-time NIDS for small networks such as a home, a hostel or a college laboratory. A Python sniffer captures live TCP, UDP and ARP packets from the Wi-Fi or Ethernet interface, a rule-based detection module written from first principles inspects them, and every confirmed threat is pushed within a second to a web dashboard that any browser on the network can open. Three attack classes are covered: TCP port scanning, TCP SYN-flood denial of service and ARP spoofing. Deterministic rules with stated thresholds are used in place of machine learning so that every alert can be traced to a specific counter and a specific packet, which is the transparency a small network administrator actually needs [13].

Before the system is described, the important technical terms used in this synopsis are defined below:

- **Packet and header:** the unit of data transferred on a network; each packet carries layered headers (Ethernet, IP, TCP or UDP) that identify addresses, ports and control flags.
- **Promiscuous mode:** an interface setting in which the network card hands every frame it receives to the software, not only the frames addressed to it.
- **TCP three-way handshake:** the SYN, SYN-ACK and ACK exchange that opens a connection [14]; a SYN that is never acknowledged leaves a half-open connection on the server.
- **Port scan:** a rapid series of connection attempts to many ports of a host, made to discover which services are running [4].
- **SYN flood:** a denial-of-service attack that sends a large number of SYN packets to exhaust the server's table of half-open connections [3], [16].
- **ARP spoofing:** sending forged ARP replies so that a victim binds the gateway's IP address to the attacker's MAC address, allowing traffic to be intercepted [8], [9].
- **Sliding window:** a counting technique that keeps only the events of the last few seconds, so that a rate is measured continuously rather than in fixed blocks.

- **WebSocket:** a persistent, two-way connection between a browser and a server, used here to push alerts the instant they are generated [12].

The system consists of four cooperating components, shown in Figure 1. The Sniffer Engine uses Scapy [1], a Python packet-manipulation library, over the Npcap capture driver [5] on Windows to read frames from the interface and decode their Ethernet, IP, TCP, UDP and ARP headers. The Threat Detection Module maintains per-source sliding-window counters and an IP-to-MAC trust table, and applies the three detection rules to every decoded packet. The Event Pipeline is a Node.js [6] server built with the Express framework that accepts alert records as JSON over HTTP POST, validates and timestamps them, and appends them to a local log file. The Real-Time Dashboard is an HTML, CSS and JavaScript page connected to the server through Socket.IO [12], a WebSocket library, so that live packet statistics and red alert banners appear without any page refresh.

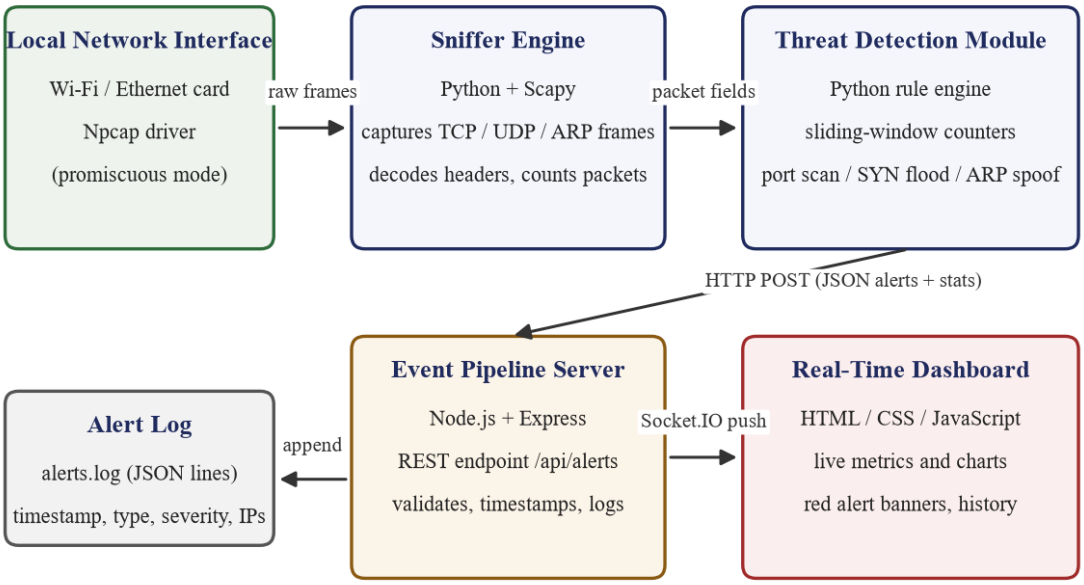


Figure 1: Architecture of the proposed real-time NIDS and the flow of packets and alerts

2. Statement of Problem with Objectives

2.1 Statement of the Problem

Small networks are the least protected part of the Internet. A home router, a hostel Wi-Fi network or a departmental laboratory rarely has any monitoring beyond the router's own log, so a port scan or an ARP-spoofing attempt can continue for its whole duration without anyone noticing. Professional tools exist, but each has limitations in this setting:

- **Packet analysers** such as Wireshark [17] capture and decode traffic in great detail, but they are designed for manual, after-the-fact analysis and do not raise alerts on their own.
- **Production NIDS engines** such as Snort [10] and Zeek [7] are powerful, but they depend on large rule sets written in their own syntax, need expert tuning, and report to text logs or separate consoles, so a beginner cannot see what is happening in real time.
- **Machine-learning prototypes** trained on datasets such as KDD Cup 99 [15] report high accuracy, but their decisions are hard to explain, they are seldom trained on the traffic of the network they protect, and they are rarely deployed in practice [13].
- **Commercial appliances** combine detection with a dashboard, but their cost places them beyond students and small offices.

There is therefore a need for a system that is light enough to run on a laptop, transparent enough that every alert can be explained line by line, and immediate enough that a threat is visible on a screen within a second of detection.

2.2 Objectives

1. To capture live TCP, UDP and ARP packets from a local Wi-Fi or Ethernet interface using Scapy and decode their protocol headers in real time.
2. To design and implement a port-scan detection rule that flags any source address which contacts more than 15 distinct destination ports on a host within a 5-second sliding window.
3. To design and implement a SYN-flood detection rule that tracks half-open TCP handshakes per destination and raises an alert when their count within the window exceeds a configurable threshold.
4. To design and implement an ARP-spoofing detection rule that maintains a trusted IP-to-MAC mapping for the gateway and known hosts and alerts on any conflicting ARP reply.
5. To build an event pipeline in which the Python modules send alerts as JSON over HTTP POST to a Node.js and Express server that validates, timestamps and logs them.
6. To develop a browser dashboard that receives live packet statistics and alerts through Socket.IO and shows them as charts, tables and red alert banners without a page refresh.
7. To evaluate the system with controlled, self-generated attack traffic on an isolated network, measuring detection latency and the false-positive rate.

3. Literature Survey

The idea of detecting intrusions by examining system activity was formalised by Denning [2], whose model compared audit records against profiles of normal behaviour and a set of rules; almost every later IDS, including this project, follows that rule-and-profile structure. Scarfone and Mell [11] later classified such systems by placement, network or host, and by method, signature or anomaly, which is the vocabulary used in this synopsis.

Roesch [10] introduced Snort in 1999 as a “lightweight” NIDS: a single-threaded engine that decodes packets and matches them against a text rule set. Snort proved that a rule-based sensor on commodity hardware could be effective, and it remains the reference against which small NIDS projects are compared. Paxson's Bro system [7], now called Zeek, moved detection into a scripting language that reacts to protocol events rather than byte patterns. Both are far larger than a small network needs, and neither ships with a real-time web view.

For port scans, Jung et al. [4] showed that a scanner can be identified quickly from the proportion of its connection attempts that fail, because legitimate clients rarely contact closed ports. The simpler rule adopted here, counting distinct destination ports per source within a short window, is the approach of Snort's scan pre-processor and is easy to explain and test. For SYN floods, Wang, Zhang and Shin [16] detected the attack from the imbalance between SYN and FIN packets, and RFC 4987 [3] documents the attack and mitigations such as SYN cookies; counting half-open handshakes in a window applies the same idea. ARP spoofing exploits the stateless, unauthenticated design of the protocol defined in RFC 826 [8]. Ramachandran and Nandi [9] reviewed passive and active detection methods; the passive method chosen here, watching for a change in the MAC address bound to a known IP address, is the principle used by the arpwatch utility.

Sommer and Paxson [13] explained why machine-learning detection, despite strong results on the KDD Cup 99 dataset analysed by Tavallaee et al. [15], is seldom used operationally: false positives are costly, decisions are hard to interpret, and training data differs from live traffic. That argument motivates the deterministic rules of this project. On the implementation side, Scapy [1] provides layer-by-layer packet dissection in Python, Npcap [5] supplies raw capture on Windows, and Node.js [6] with Socket.IO [12] gives reliable, reconnecting WebSocket communication between the server and browsers; Stallings [14] is the textbook reference for the underlying security concepts.

4. Expected Outcome / Scope of the Project

4.1 Expected Outcome

At the end of the ten-week cycle the project will deliver a working prototype with the following measurable results:

- Live capture and classification of packets on the laptop's Wi-Fi interface, with packets per second and protocol distribution reported every second.
- Correct detection of a port scan, a SYN flood and an ARP-spoofing attempt generated on an isolated test network, with the alert reaching the dashboard within about one second of the threshold being crossed.
- A persistent alert log in JSON-lines format recording the timestamp, attack type, severity, source and destination addresses and the counter value that fired the rule, so that every alert can be audited.
- A measured false-positive rate during ordinary use such as web browsing and video streaming over a fixed observation period.
- Source code, unit tests for each rule, a 40-page report with data-flow and UML diagrams, and a presentation.

4.2 Key Features and Intended Users

Key features are a Berkeley Packet Filter so that only TCP, UDP and ARP frames reach Python; rules whose thresholds and window lengths are read from a configuration file; a cool-down period that stops one attacker generating hundreds of duplicate alerts; three severity levels shown as colour-coded banners; an alert history table; and a dashboard that can be opened from any device on the same network. The intended users are students learning network security, laboratory and hostel network administrators, and home users who want visibility into their Wi-Fi network. Practical applications include monitoring a departmental laboratory during practical sessions, demonstrating attacks and their signatures in a classroom, and acting as the sensor layer of a larger monitoring system.

4.3 Scope, Limitations and Future Enhancements

The prototype monitors the traffic visible to one network interface: on a switched network this is broadcast traffic, traffic addressed to the monitoring host and anything mirrored to it, so full coverage of a large LAN would need a mirror port or several sensors. Encrypted payloads are not inspected, because all three rules work on header fields alone, and detection is limited to the three attack classes listed. The rule engine is designed so that further rules can be added as Python functions. Planned extensions are additional rules for ICMP floods, DNS tunnelling and SSH brute force; e-mail or Telegram notification; storage of alerts in SQLite; replay of saved PCAP files for offline analysis; an optional anomaly-detection module trained on the network's own baseline; and an active-response mode that adds Windows Firewall rules against a confirmed attacker, turning the IDS into an IPS.

5. Tentative Work Plan

Table 1 maps the major activities of the project to the ten-week cycle and to the evaluation milestones fixed by the department. Requirement analysis and literature study occupy the first two weeks, design the third, development weeks four to seven, testing week eight, and documentation and presentation the final two weeks.

Table 1: Week-wise work plan for the ten-week project cycle

Week	Major activities	Milestone / deliverable
1	Requirement analysis: study of IDS types, attack signatures and packet-capture mechanisms; installation of Python, Scapy, Npcap, Node.js and VS Code; verification of raw capture on the Wi-Fi interface.	Development environment ready; weekly logbook opened.
2	Literature study of Snort, Zeek, Scapy and the port-scan, SYN-flood and ARP-spoofing papers; preparation of this synopsis.	Synopsis submitted.
3	System design: architecture, module interfaces, alert JSON schema, level-0 and level-1 DFDs, use-case and sequence diagrams; selection of thresholds.	Title locked in the Title Approval Register; design document.
4	Sniffer Engine: capture loop with BPF filter, header parsing for Ethernet, IP, TCP, UDP and ARP, per-second statistics.	Live packet counters running.
5	Threat Detection Module: sliding-window structures, port-scan and SYN-flood rules, unit tests with crafted packets.	Two rules passing unit tests.
6	Event pipeline: Express POST endpoint with validation and logging; basic dashboard connected through Socket.IO.	Mid-term evaluation: at least 30% working demo (live capture and a port-scan alert on the dashboard).
7	ARP-spoofing rule with trust table; live charts, alert banners, history table and severity colours on the dashboard.	All three rules integrated end to end.
8	Integration testing on an isolated network with self-generated attack traffic; threshold tuning; measurement of detection latency and false positives.	Test report with results.
9	Draft report of 40 pages with diagrams and screenshots; plagiarism check; consolidation of the weekly logbook.	Draft report submitted; similarity below 20%.
10	Final corrections and binding; 10 to 12 slide presentation; demonstration rehearsal.	Final submission, live demonstration and viva-voce.

6. Software / Hardware Required

The project needs no special hardware. The development laptop described in Table 2 also serves as the demonstration machine, and a second device on the same Wi-Fi network is used only to generate test traffic. The software listed in Table 3 is free or open source, except for the operating system and the Microsoft Office tools used for documentation.

Table 2: Hardware requirements

Component	Minimum requirement	Development machine used
Processor	64-bit dual-core processor, 2 GHz	Intel Core i7-11800H, 8 cores, 2.30 GHz
Memory	4 GB RAM	8 GB DDR4 RAM
Storage	2 GB free disk space	512 GB NVMe SSD
Network interface	Wi-Fi or Ethernet adapter that supports promiscuous capture	Intel Wi-Fi 6 AX201
Test traffic source	Any second laptop or phone on the same Wi-Fi network	Second laptop / Android phone
Network	Wi-Fi router forming an isolated test network	Home Wi-Fi router

Table 3: Software requirements

Category	Software and version	Purpose
Operating system	Windows 11 Home, 64-bit (the code also runs on Linux)	Development and demonstration
Programming languages	Python 3.13; JavaScript (ES2020), HTML5, CSS3	Sniffer and detector; server and dashboard
Packet capture	Scapy 2.6; Npcap 1.8	Live capture and header dissection on Windows
Server runtime	Node.js 24 LTS with npm; Express 5; Socket.IO 4	Alert receiver, log writer, WebSocket push
Editor and version control	Visual Studio Code; Git and GitHub	Coding, debugging, version history
Browser	Google Chrome or Microsoft Edge	Dashboard client
Testing tools	Nmap; Wireshark; custom Scapy scripts	Generating and verifying test traffic
Documentation	Microsoft Word and PowerPoint; draw.io	Report, presentation and diagrams

7. References

[1] Biondi, P. and the Scapy community, “Scapy Documentation, Release 2.6,” 2024. Website URL: https://scapy.readthedocs.io/en/latest/ (accessed 14 September 2026).

[2] Denning, D. E., “An Intrusion-Detection Model,” IEEE Transactions on Software Engineering, Vol. SE-13, No. 2, pp. 222–232, 1987.

[3] Eddy, W., “TCP SYN Flooding Attacks and Common Mitigations,” RFC 4987, IETF, 2007.

[4] Jung, J., Paxson, V., Berger, A. W. and Balakrishnan, H., “Fast Portscan Detection Using Sequential Hypothesis Testing,” Proceedings of the IEEE Symposium on Security and Privacy, pp. 211–225, 2004.

[5] Nmap Project, “Npcap: Packet Capture Library for Windows,” 2024. Website URL: https://npcap.com/ (accessed 14 September 2026).

[6] OpenJS Foundation, “Node.js Documentation,” 2025. Website URL: https://nodejs.org/docs/latest/api/ (accessed 14 September 2026).

[7] Paxson, V., “Bro: A System for Detecting Network Intruders in Real-Time,” Computer Networks, Vol. 31, No. 23–24, pp. 2435–2463, 1999.

[8] Plummer, D. C., “An Ethernet Address Resolution Protocol,” RFC 826, IETF, 1982.

[9] Ramachandran, V. and Nandi, S., “Detecting ARP Spoofing: An Active Technique,” Proceedings of the International Conference on Information Systems Security (ICISS 2005), Lecture Notes in Computer Science, Vol. 3803, Springer, pp. 239–250, 2005.

[10] Roesch, M., “Snort – Lightweight Intrusion Detection for Networks,” Proceedings of the 13th USENIX Systems Administration Conference (LISA ’99), pp. 229–238, 1999.

[11] Scarfone, K. and Mell, P., “Guide to Intrusion Detection and Prevention Systems (IDPS),” NIST Special Publication 800-94, National Institute of Standards and Technology, 2007.

[12] Socket.IO, “Socket.IO Documentation, Version 4,” 2025. Website URL: https://socket.io/docs/v4/ (accessed 14 September 2026).

[13] Sommer, R. and Paxson, V., “Outside the Closed World: On Using Machine Learning for Network Intrusion Detection,” Proceedings of the IEEE Symposium on Security and Privacy, pp. 305–316, 2010.

[14] Stallings, W., Network Security Essentials: Applications and Standards, 6th ed., Pearson, 2017.

[15] Tavallaee, M., Bagheri, E., Lu, W. and Ghorbani, A. A., “A Detailed Analysis of the KDD CUP 99 Data Set,” Proceedings of the IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA), pp. 1–6, 2009.

[16] Wang, H., Zhang, D. and Shin, K. G., “Detecting SYN Flooding Attacks,” Proceedings of IEEE INFOCOM, Vol. 3, pp. 1530–1539, 2002.

[17] Wireshark Foundation, “Wireshark User’s Guide,” 2025. Website URL: https://www.wireshark.org/docs/wsug_html_chunked/ (accessed 14 September 2026).